# INTERNET OF THINGS (IoT) – BATCH 2

Practical Journal: Aim, Components, Connections and Program

## SLIP 1

### Practical 4: 8-LED Module

**Aim:** To interface an 8-LED module with Raspberry Pi 3A+ and control three LEDs to blink in a specific pattern using Python programming.

**Components:** Raspberry Pi 3A+, 8-LED module, jumper wires.

**Connections:**

- Pin 11 → LED 1
- Pin 13 → LED 2
- Pin 15 → LED 3
- Pin 6 → Common GND

**Program:**

```
import RPi.GPIO as GPIO
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BOARD)

leds = [11, 13, 15]

for led in leds:
    GPIO.setup(led, GPIO.OUT)
    GPIO.output(led, GPIO.LOW)

try:
    while True:
        for led in leds:
            GPIO.output(led, GPIO.HIGH)
            time.sleep(1)
            GPIO.output(led, GPIO.LOW)

except KeyboardInterrupt:
    GPIO.cleanup()
```

### Practical 11: SG90 Servo Motor

**Aim:** To interface an SG90 servo motor with Arduino and demonstrate its rotation from 0° to 180° and back to 0°.

**Components:** Arduino Uno, SG90 servo motor, jumper wires.

**Connections:**

- Signal (orange/yellow) → Pin 9
- VCC (red) → 5V
- GND (brown/black) → GND

**Program:**

```
#include <Servo.h>

Servo myservo;
int pos = 0;

void setup() {
    myservo.attach(9);
}

void loop() {
    for (pos = 0; pos <= 180; pos++) {
        myservo.write(pos);
        delay(15);
    }

    for (pos = 180; pos >= 0; pos--) {
        myservo.write(pos);
        delay(15);
    }
}
```

## SLIP 2

### Practical 5: TM1637 4-Digit Display

**Aim:** To interface a TM1637 4-digit 7-segment display with Raspberry Pi 3A+ and display the current time using Python programming.

**Components:** Raspberry Pi 3A+, TM1637 display, jumper wires.

**Connections:**

- CLK → GPIO 21
- DIO → GPIO 20
- VCC → 5V
- GND → GND

**Install library:** pip3 install tm1637

**Program:**

```
import tm1637
import time

CLK = 21
DIO = 20

display = tm1637.TM1637(clk=CLK, dio=DIO)
display.brightness(2)

try:
    while True:
        t = time.localtime()
        hour = t.tm_hour
        minute = t.tm_min
        display.numbers(hour, minute, colon=True)
        time.sleep(1)

except KeyboardInterrupt:
    display.clear()
```

### Practical 12: LM35 Temperature Sensor and Buzzer

**Aim:** To interface an LM35 temperature sensor and buzzer with Arduino and monitor temperature changes through the Serial Monitor.

**Components:** Arduino board, LM35 temperature sensor, buzzer, jumper wires.

**Connections:**

- LM35 VCC → 5V
- LM35 GND → GND
- LM35 OUT → A0
- Buzzer (+) → Pin 35 (as recorded in journal)
- Buzzer (−) → GND

**Program:**

```
int sensorPin = A0;
int buzzer = 35;

void setup() {
    Serial.begin(9600);
    pinMode(buzzer, OUTPUT);
}

void loop() {
    int value = analogRead(sensorPin);
    float voltage = value * 5.0 / 1023.0;
    float temperature = voltage * 100.0;

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");

    if (temperature > 30) {
        digitalWrite(buzzer, HIGH);
    }
    else {
        digitalWrite(buzzer, LOW);
    }

    delay(1000);
}
```

## SLIP 3

### Practical 7: CPU Usage Monitoring

**Aim:** To monitor the CPU usage of Raspberry Pi 3A+ and display the CPU utilization percentage using a Python program.

**Components:** Raspberry Pi 3A+.

**Connections:**

- No external hardware connections required.

**Install library:** pip3 install psutil

**Program:**

```
import psutil

cpu_usage = psutil.cpu_percent(interval=1)
memory_usage = psutil.virtual_memory().percent

with open("/sys/class/thermal/thermal_zone0/temp", "r") as file:
    temperature = int(file.read()) / 1000

print("CPU Usage:", cpu_usage, "%")
print("Memory Usage:", memory_usage, "%")
print("CPU Temperature:", temperature, "C")
```

### Practical 8: Active and Passive Buzzers

**Aim:** To interface active and passive buzzers with Raspberry Pi 3A+ and generate sound using Python programming.

**Components:** Raspberry Pi 3A+, active buzzer, passive buzzer, jumper wires.

**Connections:**

- GPIO 11 (Physical Pin 23) → Active buzzer (as recorded in journal)
- GPIO 9 (Physical Pin 21) → Passive buzzer (as recorded in journal)
- Physical Pin 6 → GND

**Program:**

```
from gpiozero import Buzzer
from time import sleep

buzzer = Buzzer(17)

while True:
    buzzer.on()
    sleep(1)
    buzzer.off()
    sleep(1)
```

**Important note:** Practical 12's journal specifies Arduino pin 35, which is not available on an Arduino Uno. Confirm the actual board and pin before connecting the buzzer.